

Node.js

সিনিয়র লেভেল রিভিশন — ১৫ মিনিট। PHP ব্যাকগ্রাউন্ড থেকে আসা ইঞ্জিনিয়ারের জন্য সবচেয়ে বড় পার্থক্য: single-threaded event loop মডেল — সেটাই ইন্টারভিউয়ের কেন্দ্র।

১. Event Loop কীভাবে কাজ করে?

Description: Node.js-এর সবচেয়ে জিজ্ঞাসিত প্রশ্ন। Single thread-এ হাজার concurrent connection কীভাবে — phase-গুলোসহ ব্যাখ্যা করতে পারা সিনিয়র বৈশিষ্ট্য।

৬টা Key Point:

১. মূল মডেল: এক main thread + non-blocking I/O — I/O শুরু করে দিয়ে thread পরের কাজে যায়, শেষ হলে callback queue-তে ফেরে
২. Event loop phase-চক্র: timers → pending callbacks → poll (I/O) → check (setImmediate) → close callbacks — প্রতি চক্রে ঘোরে
৩. Microtask আগে: প্রতিটা callback-এর পরে সব `process.nextTick` তারপর সব Promise callback নিঃশেষ — তারপর পরের macrotask
৪. তাই order প্রশ্নের উত্তর: sync code → nextTick → promise.then → setTimeout/setImmediate — এই ক্রম মুখস্থ
৫. I/O আসলে libuv-এর কাজ — network OS-level async, কিন্তু fs/crypto/dns libuv thread pool-এ (default 8 thread) চলে
৬. এক লাইনে সারমর্ম: "Node concurrent I/O-তে অসাধারণ, concurrent CPU-তে দুর্বল" — এই বাক্যই বাকি প্রশ্নগুলোর ভিত্তি

২. Blocking Code-এর বিপদ ও CPU-heavy কাজ সামলানো

Description: Event loop block মানে সব request জমে যাওয়া — PHP-র মতো "এক request এক process" নয়। এই পার্থক্য ধরতে পারা যাচাই করা হয়।

৬টা Key Point:

১. Main thread-এ ২০০ms-এর sync লুপ মানে ওই সময়ে সার্ভারের সব request frozen — PHP-তে এক ইউজার ভুগত, Node-এ সবাই
২. কমন অপরাধী: `JSON.parse` বিশাল payload-এ, sync fs call (`readFileSync`), জটিল regex (ReDoS), বড় লুপ/sort
৩. সমাধান ১: worker_threads — CPU-heavy কাজ (report হিসাব, image processing) আলাদা thread-এ, main loop মুক্ত
৪. সমাধান ২: কাজটা আলাদা process/queue-তে পাঠানো (BullMQ + worker process) — architecture-লেভেল উত্তর, বেশিরভাগ ক্ষেত্রে এটাই ঠিক
৫. বড় ডেটা ভাঙা: `setImmediate` দিয়ে chunk-এ চালিয়ে ফাঁকে ফাঁকে loop-কে শ্বাস দেওয়া — মাঝারি ক্ষেত্রের কৌশল
৬. ধরার উপায়: event loop lag metric মনিটর করা (`monitorEventLoopDelay`) — lag বাড়া মানে কোথাও কেউ block করছে

৩. Promise, async/await ও Error Handling

Description: Async error ঠিকমতো ধরা Node-এ বেঁচে থাকার দক্ষতা — unhandled rejection-এ প্রোডাকশন process মরে যেতে পারে।

৬টা Key Point:

1. `async/await` = Promise-এর syntax sugar — নিচে একই জিনিস; `async function` সবসময় Promise ফেরায়
2. `try/catch` শুধু `await`-করা error ধরে — `await` ভুলে গেলে rejection নীরবে পালায় (floating promise); সবচেয়ে কমন বাগ
3. Parallel vs sequential সচেতনভাবে: independent কাজ `Promise.all` — লুপে `await` দিয়ে অকারণ serial করা কমন performance ভুল
4. `Promise.all` এক fail-এ সব reject; আংশিক ব্যর্থতা মানতে হলে `Promise.allSettled` — প্রতিটার status আলাদা পাওয়া যায়
5. Express (v4)-এ `async handler`-এর error নিজে `next(err)`-এ পাঠাতে হয় (wrapper দিয়ে) — না করলে request ঝুলে থাকে; Express 5/Fastify-তে built-in
6. শেষ রক্ষা: `unhandledRejection/uncaughtException` হকে log করে `graceful shutdown` — unknown state-এ চালিয়ে যাওয়ার চেয়ে restart ভালো (process manager তুলবে)

8. Streams ও বড় ডেটা Processing

Description: ২ GB-র CSV process করতে হবে — পুরোটা মেমোরিতে তুলবেন না। Stream-চিন্তা Node-এর অন্যতম শক্তি, বাস্তব অভিজ্ঞতা এখানে ধরা পড়ে।

৬টা Key Point:

1. চার প্রকার: Readable, Writable, Duplex, Transform — ডেটা chunk-এ-chunk-এ বয়, মেমোরিতে পুরোটা কখনো থাকে না
2. `pipeline()` ব্যবহার করা (`pipe()` নয়) — error propagation আর cleanup automatic; pipe-এ error handling ফাঁকা থেকে যায়
3. Backpressure built-in: ধীর destination হলে source আপনিই থামে — নিজে stream চালালে `write()` false ফেরত সম্মান করা
4. বাস্তব উদাহরণ বলা: CSV import — fs read stream → csv-parse (Transform) → batch DB insert; মেমোরি ফ্ল্যাট থাকে
5. HTTP response-ও stream: বড় ফাইল `res`-এ pipe — S3 থেকে client পর্যন্ত সার্ভারের মেমোরি না ভরিয়ে
6. আধুনিক টাচ: async iterator দিয়ে stream পড়া (`for await (const chunk of stream)`) — পরিষ্কার syntax, একই লাভ

৫. Node.js Scale করা: Cluster, PM2, Horizontal Scaling

Description: Single thread মানে এক core — বাকি core আর একাধিক সার্ভারে কীভাবে ছড়াবেন, প্রোডাকশন অপারেশনের প্রশ্ন।

৬টা Key Point:

1. এক Node process ≈ এক CPU core — ৮ core মেশিনে ৮টা process চালানো (cluster module বা PM2 cluster mode)
2. PM2 দায়িত্ব: process per core, crash-এ auto-restart, zero-downtime `reload`, log/metrics — Node-এর Supervisor+deploy টুল
3. Container জগতে উল্টো নিয়ম: এক container এক process, replica দিয়ে scale — PM2-cluster তখন দরকার নেই

4. Horizontal scale-এর শর্ত PHP-র মতোই: stateless — session/cache Redis-এ; in-memory state রাখলে LB-র পেছনে ভাঙবে
5. WebSocket scale-এর বিশেষত্ব: connection sticky বা shared pub/sub লাগে — Socket.IO + Redis adapter দিয়ে সব instance-এ event ছড়ানো
6. Graceful shutdown সঠিকভাবে: SIGTERM ধরে নতুন request নেওয়া বন্ধ → চলমানগুলো শেষ → DB connection বন্ধ → exit — rolling deploy-এ request না হারানোর চাবি

৬. Express/NestJS: API Architecture

Description: Node-এ backend structure কীভাবে সাজান — unopinionated Express-এ শৃঙ্খলা আনা, বা NestJS-এর opinionated কাঠামো। Laravel-অভিজ্ঞের জন্য তুলনাটা সহজ অস্ত্র।

৬টা Key Point:

1. Express minimal — structure নিজের দায়িত্ব: route → controller → service → data layer ভাগ করা; সব logic route callback-এ ঢালা জুনিয়র লক্ষণ
2. Middleware chain Laravel-এরই ধারণা: auth, validation, logging — `(req, res, next)`; error middleware-এর signature আলাদা (৪ argument)
3. Centralized error handler + custom error class (statusCode সহ) — প্রতিটা controller-এ try/catch-toast ছড়ানো নয়
4. Validation boundary-তে: Zod/Joi দিয়ে request schema — ভেতরের কোড trusted input পায় (Laravel FormRequest-এর সমতুল্য)
5. NestJS = Node-এর "Laravel/Spring": DI container, module, decorator, guard/pipe — বড় টিমে শৃঙ্খলা ফ্রি; ছোট সার্ভিসে overhead
6. তুলনা-লাইন ইন্টারভিউতে: "Laravel-এ framework সিদ্ধান্ত নিয়ে রাখে, Express-এ আমি নিই — তাই Express-এ discipline-টা টিমের convention থেকে আসতে হয়"

৭. Node.js Security Essentials

Description: API-র কমন দুর্বলতা আর Node-বিশেষ ঝুঁকি — dependency chain থেকে input validation পর্যন্ত।

৬টা Key Point:

1. সবচেয়ে বড় Node-ঝুঁকি supply chain: হাজার transitive dependency — `npm audit`/Snyk, lockfile commit, সন্দেহজনক package এড়ানো
2. Input validation সব জায়গায়: SQL injection (parameterized query/ORM), NoSQL injection (`{ $gt: '' }` MongoDB-তে) — body সরাসরি query-তে নয়
3. Auth token: JWT হলে ছোট expiry + refresh, secret শক্ত, `alg: none` জাতীয় লাইব্রেরি বাগ সচেতনতা; sensitive হলে httpOnly cookie
4. Rate limiting + brute force protection (Redis-backed counter) — login/OTP endpoint-এ বাধ্যতামূলক
5. Security header (helmet), CORS সচেতনভাবে সীমিত (wildcard + credentials কখনো নয়), payload size limit
6. Secret management: `.env` git-এর বাইরে, প্রোডাকশনে vault/SSM; error response-এ stack trace leak না করা

৮. Node.js Testing ও Debugging

Description: Async-heavy কোড টেস্ট ও ডিবাগ করার কৌশল — memory leak খোঁজা সহ প্রোডাকশন debugging অভিজ্ঞতা।

৬টা Key Point:

1. টেস্ট স্ট্যাক: Jest/Vitest + supertest (HTTP endpoint-এ আসল request) — unit-এ service logic, integration-এ route+DB
2. Async টেস্টে await/return ভুলে গেলে টেস্ট "pass" করে আসলে কিছুই চেক না করে — সবচেয়ে ধোঁকাবাজ বাগ
3. External dependency mock: DB/HTTP call —nock/msw দিয়ে HTTP stub; টেস্ট নেটওয়ার্কে গেলে সেটা টেস্ট না, জুয়া
4. Debugging: `node --inspect` + Chrome DevTools — breakpoint, CPU profile; console.log-এর ওপরে উঠতে পারা
5. Memory leak খোঁজা: heap snapshot তুলনা (আগে/পরে) — কমন কারণ: global cache বেড়েই চলা, event listener remove না করা, closure-এ বড় object আটকে থাকা
6. প্রোডাকশন: heap/event-loop-lag metric + `--max-old-space-size` সচেতনভাবে — OOM kill-এর আগে সংকেত পাওয়া

৯. TypeScript Node Backend-এ

Description: এখন কার্যত স্ট্যান্ডার্ড — কেন ব্যবহার করবেন আর runtime সীমাবদ্ধতাটা কোথায়, দুটোই বলতে হয়।

৬টা Key Point:

1. মূল লাভ backend-এ: refactor সাহস, API contract-এ type, IDE intelligence — বড় কোডবেসে ভুলের শ্রেণিটাই মুছে যায়
2. সবচেয়ে গুরুত্বপূর্ণ সত্য: type শুধু compile-time — runtime-এ কিছুই চেক হয় না; API input-এ তাই Zod-জাতীয় runtime validation লাগবেই
3. Zod প্যাটার্ন: schema একবার লিখে `z.infer`-এ type ফ্রি — validation আর type এক উৎসে, drift নেই
4. `strict: true` না হলে TypeScript অর্ধেক মিথ্যা — `any` ছড়ানো codebase-এ type-এর আশ্বাস ভুয়া; `unknown` + narrow করা অভ্যাস
5. Generic-এর ব্যবহার জানা: typed repository/response wrapper (`ApiResponse<T>`) — utility type (Partial, Pick, Omit) দৈনন্দিন অস্ত্র
6. Build pipeline সচেতনতা: tsc/esbuild → dist; দ্রুত dev-এ tsx/ts-node — আর ESM vs CommonJS-এর ব্যথা এখনো বাস্তব, এক লাইনে স্বীকার করা

১০. Node.js vs PHP/Laravel — কখন কোনটা?

Description: দুই স্ট্যাকেই কাজ করা কারো কাছে এই তুলনা প্রায় নিশ্চিত প্রশ্ন — দলীয় ঝগড়া নয়, workload-ভিত্তিক পরিণত উত্তর।

৬টা Key Point:

1. Execution মডেলই মূল পার্থক্য: PHP-FPM request-per-process (isolation ফ্রি, লিক নেই) vs Node long-running event loop (state থাকে, leak সম্ভব, কিন্তু I/O concurrency সম্ভা)
2. Node জেতে: real-time (WebSocket/chat/live tracking), অনেক concurrent I/O (gateway, proxy), frontend-এর সাথে এক ভাষা/টিম
3. Laravel জেতে: CRUD-heavy business app দ্রুত নামানো — auth, queue, ORM, admin সব batteries-included; BD মার্কেটে টিম পাওয়াও সহজ
4. CPU-heavy কাজে দুটোই দুর্বল — দুই ক্ষেত্রেই queue + আলাদা worker-এ পাঠানো; পার্থক্য হলো Node-এ block করলে ক্ষতি সর্বজনীন
5. Long-running হওয়ায় Node-এ in-process cache/connection reuse ফ্রি — PHP-তে সেটা Redis/OPcache দিয়ে পুষিয়ে নেয় (Octane-ও একই দিকে)
6. পরিণত উত্তর-লাইন: "স্ট্যাক না, সমস্যা আগে — team skill + workload + ecosystem মিলিয়ে বাছি; এক প্রোডাক্টে দুটোও চলে (Laravel core + Node real-time service)"